

Human Pose Estimation

Human Pose Estimation

1. Human Pose Estimation

1.1. Overview

1.2. Principle

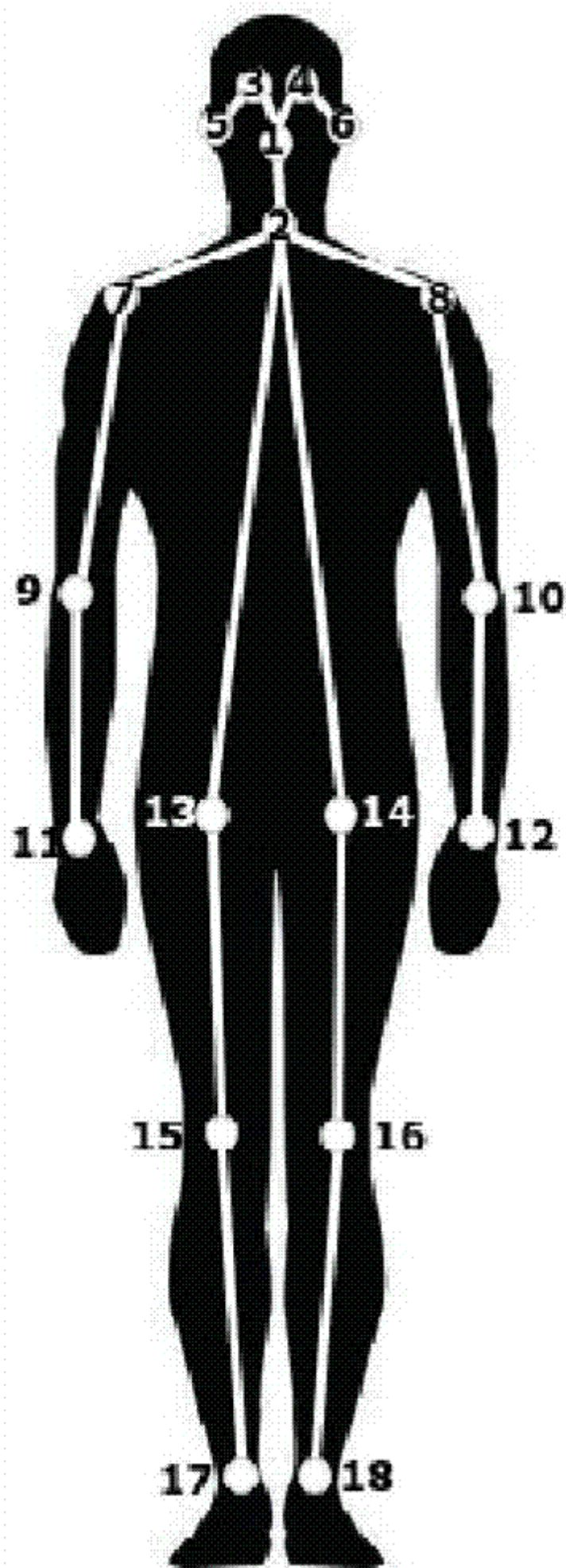
1.3. Startup

1. Human Pose Estimation

Source code path: /home/pi/yahboomcar_ws/src/yahboomcar_visual/detection

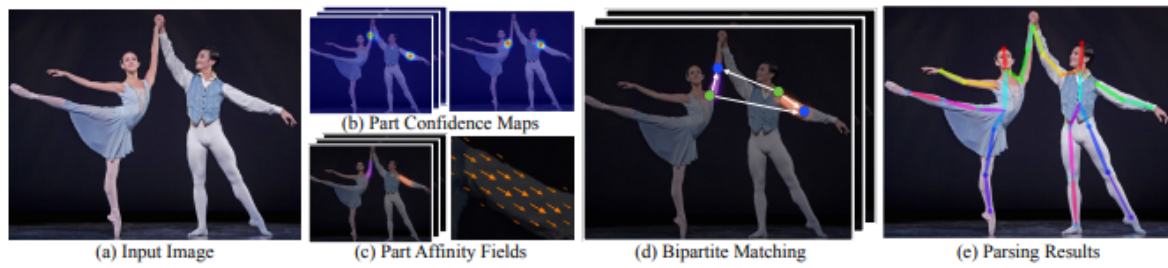
1.1. Overview

Human Pose Estimation is the process of correctly connecting the human keypoints that have been detected in an image to estimate the human pose. Human keypoints usually correspond to joints on the human body that have a certain degree of freedom, such as the neck, shoulders, elbows, wrists, waist, knees, and ankles, as shown in the figure below.



- 1 鼻子
- 2 脖子
- 3 左眼
- 4 右眼
- 5 左耳
- 6 右耳
- 7 左肩
- 8 右肩
- 9 左肘
- 10 右肘
- 11 左腕
- 12 右腕
- 13 左髓
- 14 右髓
- 15 左膝
- 16 右膝
- 17 左踝
- 18 右踝

1.2. Principle



An image is input, and features are extracted through a convolutional network to obtain a set of feature maps. Then it splits into two branches, which use CNN networks to extract Part Confidence Maps and Part Affinity Fields respectively. After obtaining these two pieces of information, we use Bipartite Matching from graph theory to compute Part Association, connecting the joint points of the same person. Due to the vector nature of PAF itself, the resulting bipartite matching is very accurate, and finally it is merged into the overall skeleton of one person. Finally, based on PAFs, Multi-Person Parsing is computed - the Multi-person parsing problem is converted into a graph problem - Hungarian Algorithm (the Hungarian algorithm is the most common algorithm for bipartite matching. The core of this algorithm is to find augmenting paths. It is an algorithm that uses augmenting paths to find the maximum matching of a bipartite graph).

1.3. Startup

```
cd /home/pi/yahboomcar_ws/src/yahboomcar_visual/detection
python target_detection_CSI.py (CSI camera)
python target_detection_USB.py (USB camera)
```

After clicking the image frame, press the [f] key on the keyboard to switch target detection.

```
if action == ord('f') or action == ord('F'):state = not state # function switch
```

Input image



Output image

84.90ms

